



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

---

---

ФАКУЛЬТЕТ \_\_\_\_\_ «Информатика и системы управления»

КАФЕДРА \_\_\_\_\_ «Теоретическая информатика и компьютерные технологии»

**Летучка № 8**  
**по курсу «Языки и методы программирования»**  
**«С++. Статические поля. new и delete»**

Студент группы ИУ9-22Б  
**Булдаков А. С.**

Преподаватель  
**Посевин Д. П.**

*Москва 2026*

## Содержание

|                            |   |
|----------------------------|---|
| Цель работы .....          | 1 |
| Класс Point .....          | 1 |
| Реализация класса .....    | 2 |
| Конструктор .....          | 2 |
| Деструктор .....           | 2 |
| Метод print() .....        | 3 |
| Класс Universe .....       | 3 |
| Реализация .....           | 3 |
| Конструктор .....          | 3 |
| Деструктор .....           | 3 |
| main() .....               | 4 |
| Компиляция и запуск .....  | 4 |
| Makefile .....             | 4 |
| Компиляция и запуск .....  | 5 |
| Ключевые понятия C++ ..... | 5 |
| Вывод .....                | 5 |

## Цель работы

Изучить основы объектно-ориентированного программирования на C++.

Реализовать классы Point и Universe с использованием статических полей, конструкторов, деструкторов и динамического выделения памяти.

### Задачи:

- Реализовать класс Point со статическим счетчиком.
- Использовать список инициализации конструктора.
- Реализовать виртуальный деструктор.
- Управлять динамической памятью с new и delete.

## Класс Point

Класс Point представляет точку на плоскости с координатами x и y.

## Реализация класса

main.cpp

```
class Point {
private:
    static int counter;

public:
    int id;
    double x, y;

    Point();
    virtual ~Point();

    void print();
};

int Point::counter = 0;
```

Статическое поле counter хранит количество созданных объектов Point и увеличивается при каждом создании.

## Конструктор

main.cpp

```
int myrand() { return rand() % 20; }

Point::Point() : id(counter++), x(myrand()), y(myrand()) {
    std::cout << "Created new Point(" << x << ' ' << y << ") id = "
<< id << "\n";
}
```

Список инициализации: id(counter++) инициализирует id значением counter и затем увеличивает counter.

## Деструктор

main.cpp

```
Point::~~Point() { cout << "Point " << id << " selfdestructed!!!!\n"; }
```

Виртуальный деструктор гарантирует правильное освобождение памяти при удалении через указатель на базовый класс.

## Метод print()

main.cpp

```
void Point::print() { std::cout << "Point (" << x << ' ' << y << ")  
id = " << id << "\n"; }
```

## Класс Universe

Класс Universe представляет набор точек.

### Реализация

main.cpp

```
class Universe {  
public:  
    int size;  
    Point* points;  
  
    Universe();  
    virtual ~Universe();  
};
```

Поле points является указателем на массив объектов Point.

## Конструктор

main.cpp

```
Universe::Universe() : size(myrand()), points(new Point[size]) {  
    std::cout << "Universe created!\n";  
}
```

Динамическое выделение памяти: new Point[size] создает массив из size объектов Point.

## Деструктор

main.cpp

```
Universe::~~Universe() {  
    for (int i = 0; i < size; i++) {  
        points[i].~Point();  
    }  
    delete[] points;  
}
```

Явный вызов деструктора для каждого объекта и освобождение памяти с `delete[]`.

## main()

main.cpp

```
int main() {
    Universe Andrey_Kabanov;

    std::cout << "\n";
    for (int i = 0; i < Andrey_Kabanov.size; i++) {
        Andrey_Kabanov.points[i].print();
    }
    std::cout << "\n";
}
```

## Компиляция и запуск

### Makefile

Makefile

```
.PHONY: all run clean

all:
    g++ -o a.out main.cpp -std=c++17

run: all
    ./a.out

clean:
    rm -f a.out
```

## Компиляция и запуск

terminal

```
$ make
$ make run
Created new Point( ... id = 0)
Created new Point( ... id = 1)
...
Universe created!

Point ( ... id = 0)
Point ( ... id = 1)
...
Point 0 selfdestructed!!!!
Point 1 selfdestructed!!!!
...
```

## Ключевые понятия C++

1. Статическое поле — поле, общее для всех объектов класса
2. Список инициализации — инициализация полей в конструкторе до выполнения тела
3. Виртуальный деструктор — обеспечивает правильное удаление производных классов
4. new и delete — выделение и освобождение динамической памяти
5. new[] и delete[] — работа с массивами

## Вывод

В ходе лабораторной работы были изучены основы C++:

1. Статический счетчик counter для учета созданных объектов
2. Список инициализации конструктора для эффективной инициализации полей
3. Виртуальный деструктор для правильного управления памятью
4. Динамическое выделение памяти с new и освобождение с delete

Эти концепции являются фундаментом объектно-ориентированного программирования на C++.